1. Architecture Redesign

Do you want to change the technology stack? Or old architecture?

Ask for you're the challenges you have in considered in your mind => Do you want to use Microservices architecture? Or your current monolithic architecture? => **Microservices Integration implementation through messaging and SNS & SQS (Subscribe and consuming)**

2. Architecture style

Unexperienced engineers believe Microservices is the best approach/answer! In fact this is not always true. You should consider lots of architecture styles and parameters such as product complexity, teams' skills/expertise and so on that help you to select the architecture style.

So for some projects, Microservices architecture is wrong approach! And using Microservices doesn't resolve all problems.

3. Microservices architecture

What do you want to do? Keep current monolithic architecture or replace all/some of application with Microservices architecture? => Redesign to cloud means function-based cloud architecture. So you have to use Microservices approach.

4. Microservices Design important Challenges

A. Microservice Granularity (Biggest challange): Software architectures usually make mistake about the name of Microservice! They realize that a service should be very small/micro! => Micro is a trap and confusing term for designing architecture.

We can't say final answer in start point, so we would try to consider/detect/recognize the service's responsibilities and granularity then we can start by designing a service and after couple of months we decide to brake it to two separated services so I decide to separate these microservices. It means changing the design along with development.

B. Services Isolation (Logic & Data):

Sharing data and logic is the most important/sensitive issue in microservices design! And leads to define **Service Granularity.**

We should design services in which each service be fully separated or services don't share data and business logic!

B.1. Business logic level Isolation:

Put a red line around each service for preventing logic sharing! Don't reuse the service logic within other services!!

If you realize two services have same logic, probably they have been a service and you consider them as on service!

Exp. Order business service has a processing logic that shouldn't be reused in other services

B.2. Data level Isolation:

We shouldn't use shared data such as shared table during mircoservice design. Each service has own separated private data and tables.

If your service needs to use other service's data you should duplicate the required data in your service database and tables. So we have lots of data redundancy (data duplication) in microservices.

We wave/accept data redundancy (data duplication) due to decrease the services coupling/dependency and prevent side effects between services (service doesn't impact on other services in features), means each service is fully independent.

B.2.1 Conceptual Data Redundancy

Whenever data redundant across services in fact data nature would be changed!

Exp. In an educational system you can consider a financial/accounting department/class for processing student tuition also an academic department/class for registration and a customer relationship management (CRM) system for marketing.

So we have:

- academic microservice
- financial/accounting microservice
- CRM microservice

Important discussion: We have only a student database table for using by all these academic, financial and CRM departments. This is wrong approach in microservice design!

- The name of student entity is considered as "student" only in academic system
- But in financial/accounting system, name of student entity is considered as "counterparty"
- "Lead" or "Customer" in CRM system

Entity's concept

So the concept of student entity is different in financial/accounting system and academic system therefore the student's properties/attributes are different in data aspect. It means along with entity movement through systems its concept/sate is changing and we have technical properties/attributes in each system.

Finally Data Redundancy means only some attributes are common/shared between systems/services such as student name, Social security number and we accept these data redundancy and don't use the shared data for achieving the systems independency and decoupling.

Sharing data and logic is the most important/sensitive issue in microservices design! And leads to define **Service Granularity.**

C. Applications (Microservices) Integration/cohesion or relationship management (Microservices are working together)

So far we detect/design systems/micoservices but our application is not a set of separated islands! The flow diagram for each entity should use the initial data in start state/system/CRM and we don't want to enter data for each state/system/microservices! And each entity/student object automatically should be dispatch from CRM to Academic for **registration** and after **final registration automatically** a financial document/invoice should be created for payment.

All these processes bounded/chained to each other thereby we have to resolve this cohesion/integration challenge as well.

Resolve cohesion challenge: Microservices Integration implementation through messaging and SNS & SQS (Subscribe and consuming)

C.1. Messaging (Subscribe and consuming integration)

Focus on Event driven concept. And integration is subscribing to messages for event consuming.

Messaging is asynchronous and we only send the message and don't wait for receiver acknowledge signal. Because microservices don't aware each other in microservice architecture.

Integration establish: Each interested microservice that asks/uses data should subscribe for the data event topic (message) and after triggering the origin data event, they will receive/consume the data in SQS.

Exp.

CRM: take a new customer/lead => contract event => subscribed services

Academic: subscribed for using this new student data => get the new contract message => semester credit event => subscribed services

Financial: .....

C.2. Implement messaging with DynamoDB stream processor or hard coded component

DynamoDB stream processor will cascade the table changes to SQS Queue or SNS topics.

You can implement messaging and integration with coding a component or use the DynamoDB stream processor

C.3. Message based/asynch is not atomic or transactional

Best messaged based/asynch outcome/defect is that we don't have any **transaction** unlike monolithic systems (atomic or has transaction).

Exp. After getting a student in CRM system with new contract we can't guarantee for academic registration contract!

This is a **defect** of messaged based/asynch system and we can't guarantee the transaction execution but this is eventually consistent because of high reliable SQS and SNS.

Exp. The academic service might be offline and after system turning on the message will be consume.

C.4. Transactional

If we want to use **transaction** we have to implement complicated processes or handle it manually (lots of headache).

Exp.

CRM: take a new customer/lead => contract event => Academic

Academic: but student resign from semester credits => contract cancel event => CRM

5. DevOps Multi Environment

5.1 Infrastructure as Code

We don't create any resources manually! => Infrastructure as Code (IaC) is part of project and CodePipeline => cdk-infrastructure-pipeline => after each code merge/push/publish => IaC CodePipeline is executed then provisions the AWS resources => finally we deploy the backend code

What is your Infrastructure as Code tools? I used CDK with typescript, AWS Cloudformation

5.2 Infrastructure as Code tools approaches

1- General cross platform tools
   Like Terraform that works with Azure or GCP providers => support all clouds => but don't have all AWS CDK features and flexibilities and would be updated by AWS after adding a new service or any update
2- Cloud private tools
   AWS CDK or Azure

5.3 Environments

Infrastructure as Code (IaC) => cdk-infrastructure-pipeline => Config.ts => *PIPELINE_CONFIG* => *waves*

Create different stages/environments in wave for implementing and integrating reliability in testing, qa and UAT environments => after staging and testing in these environments => finally we have reliability in production

5.3.1 Git Flow will directly connects to environments

What is the development flow in git and branching? How many branches do you have? Master, dev, int

Developer creates their branches? Then merge it with development branch or master?

If merge it with development branch => when development branch merged to master? or when is release?

Before production => we can have only one branch

Project in production (after production) => we need 2 branches

1- Development branch
   Latest changes made by developer team are merged

2- Master branch (deployed to production)
   Master branch is always updating from development branch in two ways
   A- Patch/warranty/Bug fix: after a bug resolving (patch) immediately go to prod => code changes will be applied to a Master copy because development branch is under coding and it might have bugs and note ready for release => after successful Patch/warranty/Bug fix release (production) => merge the Master copy code changes to Master

   B- Release: cascade/merge development branch new changes to master (release) => first of all we should merge latest Master code to development branch (update development branch from Master and fixed bugs in Master)

5.3.2 Creating environments with waves

The challenge is how many pipeline shall we have? => After first release we created 2 branches: Master and Development

Master branch => trigger a CodePipeline for only production/prod

Development branch => trigger a CodePipeline for other lower environments/nonprod (dev, test, qa, uat)

How many environment do you have?

If you have only two prod and nonprod environments?

How many nonprod environments (lower sandboxes: test, dev, qa, uat) do you have? And how the connected to code branches?

=> do you have one CodePipeline for all nonprod environments? And one separated CodePipeline for prod?

Or you have only one CodePipeline with approval step for transition between environments?

Development branch => code changes merged/committed => CodePipeline deploys to dev => CodePipeline deploys to test => then deploy to qa => but for uat you need to hit the review button for transit to uat environment

6. Automation Test (QA automation) Level

What level of automation test do you have? And how do you perform your tests?

We should automate everything in organization

We shouldn't test our code manually! It's costly => QA automation

6.1 Testing levels: A Complete Approach to Quality Assurance (QA)

https://testsigma.com/blog/levels-of-testing/

6.1.1 Unit Test

We test only a specific function (not the system) in **unit test** so we disconnect its connections from everywhere => in **.Net** with dependency injection we **disconnect its connections** but in JavaScript it's different.

Unit test doesn't show the **integration** of functions is working well.

Unit Test **only coverage** but **doesn't guarantee** the **quality of product,** even **%100 of test are passed!** It **only shows a level of quality** but we **can't say the product/system working without defects.**

Unit Test: Because we **disconnected the dependencies** during **unit testing** otherwise it's not a unit test

Unit Test coverage should be very good and more than %90, %80 or %70 on their code

C# => X-Unit JavaScript => Jest

6.1.2 Integration (Intern) Test

Integration test shows **the integration of functions is working well** => Intern Test: **UI test**

https://www.sitepen.com/blog/what-makes-intern-different

What are the tools you are using for integration (intern) test?

We already used **WebdriverIO** that based on **Selenium**

https://webdriver.io/ https://webdriver.io/docs/why-webdriverio

Now we're using **Playright** (based on **Selenium**) from Microsoft

https://playwright.dev/

Selenium => for testing a web page we find the page HTML elements (buttons, textboxes) => then in test we say fill the textbx1 with value1 or click on button1 => we **declare** and Selenium **performs automatically (UI test**)

6.1.3 Acceptance Test (End to end Test or User acceptance test)

https://testsigma.com/blog/levels-of-testing/